Wielomian Wilkinsona: Studium Niestabilności

Arytmetyka Komputerowa - Metody Obliczeniowe

Katarzyna Piskorz

WI AGH

11.03.2026

Nasze oczekiwania

Zanim przystąpimy do analizy, określmy teoretyczne wartości wielomianu:

$$W(x) = (x-1)(x-2)\dots(x-20)$$

Z definicji oczekujemy, że:

- Pierwiastkami wielomianu będą dokładnie liczby całkowite od 1 do 20. Zatem wykres pierwiastków na płaszczyźnie zespolonej powinien leżeć wyłącznie na osi liczb rzeczywistych.
- Wartość wyrażenia dla $x = 1$ powinna wynosić dokładnie 0.
- Wartość wyrażenia dla $x = 20$ powinna wynosić dokładnie 0.

Środowisko Obliczeniowe

Do przeprowadzenia eksperymentu wykorzystano zdefiniowane środowisko:

- **Sprzęt:** Apple Mac (Procesor M2, 16 GB RAM)
- **Architektura:** ARM64 (Apple Silicon)
- **System operacyjny:** macOS 15.7.4
- **Kompilator C++:** Apple Clang 17.0.0
- **Interpreter Pythona:** Python 3.14.2 (wykorzystano domyślny 64-bitowy typ `float`, odpowiadający `double` w języku C)
- **Zależności:** Implementacja własna (Schemat Hornera) oraz biblioteka NumPy (do wyznaczania wektora pierwiastków i symulacji błędów)

Gigantyczne współczynniki postaci rozwiniętej

Przejście z postaci iloczynowej do ogólnej $W(x) = \sum_{i=0}^{20} a_i x^i$ drastycznie zmienia rzędy wielkości parametrów, co prezentuje Tabela 1.

Tabela 1: Wybrane współczynniki postaci rozwiniętej wielomianu Wilkinsona.

Współczynnik	**Dokładna wartość**
a_{20}	1
a_{19}	−210
a_{18}	20 615
...	...
a_{15}	−1 672 280 820
...	...
a_2	11 803 759 753 640 704 000
a_1	−8 752 948 036 761 600 000
a_0	2 432 902 008 176 640 000

Wniosek: Wyraz wolny $a_0 = 20!$ jest liczbą 19-cyfrową. Zmienna 64-bitowa (ok. 15-17 cyfr znaczących mantysy) fizycznie nie potrafi zachować tej precyzji, co prowadzi do błędów zaokrągleń już na etapie zapisu.

Implementacja: Schemat Hornera w Pythonie

Zaimplementowany algorytm wyznacza zaburzone współczynniki, a następnie oblicza wartość wielomianu.

Listing 1: Generacja współczynników i schemat Hornera

```
def get_wilkinson_coeffs(n):
    coeffs = [1.0] # Wymuszenie precyzji float64
    for r in range(1, n+1):
        new_coeffs = [0.0] * (len(coeffs) + 1)
        for i in range(len(coeffs)):
            new_coeffs[i] += coeffs[i] * (-r)
            new_coeffs[i+1] += coeffs[i]
        coeffs = new_coeffs
    return coeffs

def horner(coeffs, x):
    result = 0.0
    for i in range(len(coeffs) -1, -1, -1):
        result = result * x + coeffs[i]
    return result
```

Wyniki obliczeń (Schemat Hornera)

W Tabeli 2 zestawiono wartości wyliczone ze schematu sumacyjnego dla postaci rozwiniętej wielomianu w porównaniu z wartościami dokładnymi.

Tabela 2: Wartości wielomianu Wilkinsona wyliczone schematem Hornera dla typu 64-bitowego (Python).

Badany punkt x	**Oczekiwane**	**Wynik numeryczny (64-bit)**
$x = 1.0$	0	1024.0
$x = 2.0$	0	8192.0
$x = 19.0$	0	$-16\ 070\ 033\ 408.0$
$x = 20.0$	0	**-27 029 504 000.0**

Mimo użycia stabilnego matematycznie algorytmu, kumulacja błędów zmiennoprzecinkowych dla $x = 20$ generuje błąd rzędu **27 miliardów**.

Wpływ precyzji i architektury sprzętowej (C++)

Aby zweryfikować zachowanie algorytmu w różnych formatach precyzji, pomiary powtórzono w języku C++. Wyniki zestawiono w Tabeli 3.

Tabela 3: Błędy schematu Hornera w zależności od typu deklarowanej pamięci w C++.

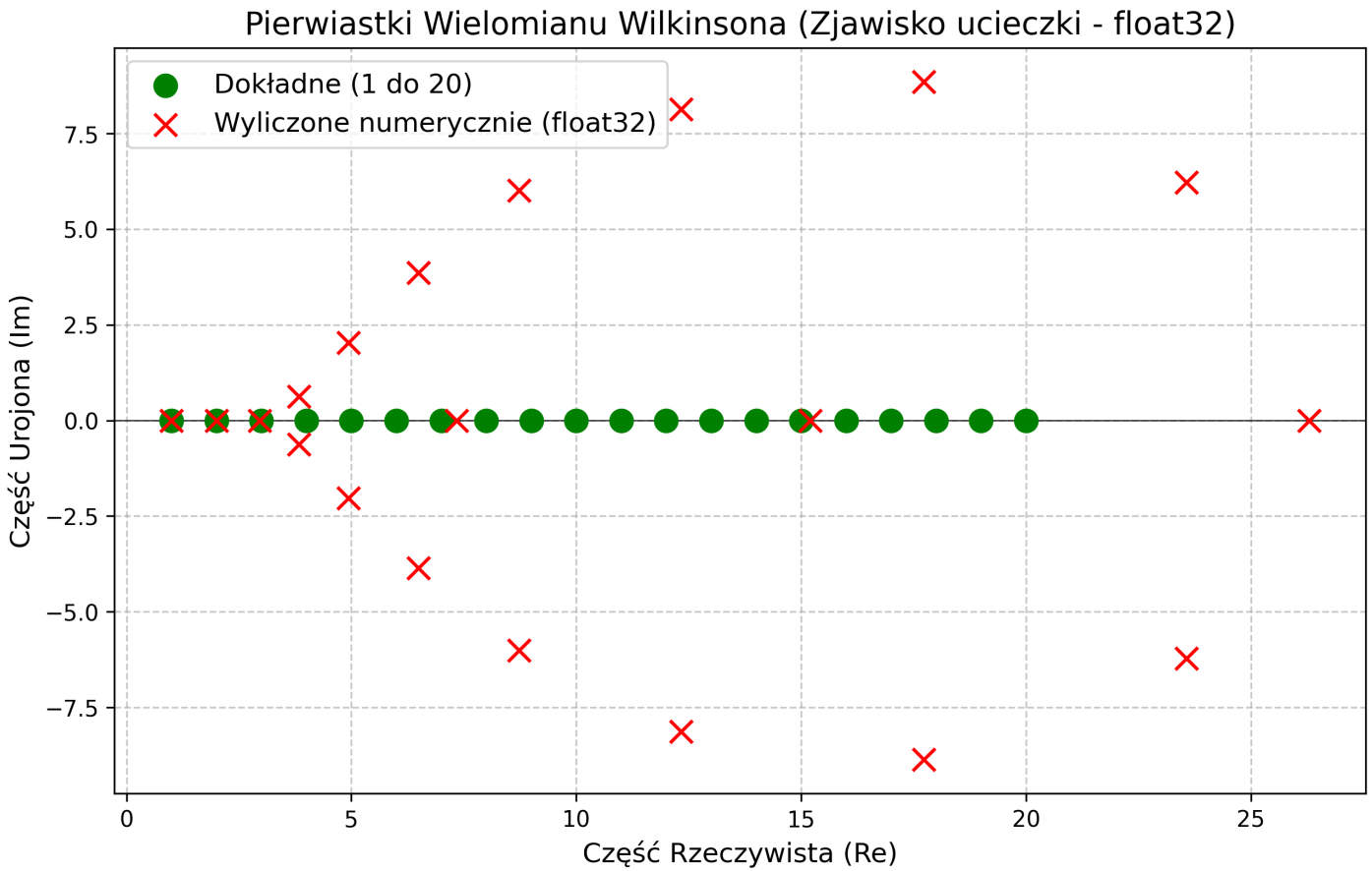
Typ (Precyzja)	**Wynik dla W(1)**	**Wynik dla W(20)**
`float` (32-bit)	0.0	-1.52×10^{21}
`double` (64-bit)	1024.0	-1.22×10^{10}
`long double`	1024.0	-1.22×10^{10}

Obserwacje sprzętowe:

- Typ `float` (32-bit) generuje skrajną utratę precyzji (błąd rzędu 10^{21}).
- W specyficznej **architekturze ARM64** (Apple Silicon), kompilator utożsamia `long double` z typem `double` (64-bit). Wynika to z braku sprzętowego wsparcia dla formatu 80-bitowego (typowego dla układów x86_64 Intela).

Ucieczka pierwiastków na płaszczyźnie zespolonej

Zjawisko utraty precyzji wizualizuje Rysunek 1, przedstawiający numeryczne wyznaczenie pierwiastków z zaburzonej (32-bitowej) postaci rozwiniętej.



Rysunek 1: Rozkład wyliczonych numerycznie pierwiastków wielomianu Wilkinsona po symulacji ucięcia precyzji bitowej.

Analiza numeryczna wykresu

Na podstawie Rysunku 1 oraz danych wyjściowych obserwujemy, że:

- **Początkowa stabilność:** Małe pierwiastki (od $x = 1$ do $x = 3$) pozostają odporne na błędy zaokrągleń współczynników.
- **Kulminacja błędu (Pary sprzężone):** Zjawisko "zderzania się"zaburzonych pierwiastków na osi rzeczywistej powoduje ich wypychanie w dziedzinę liczb zespolonych. Występują one w postaci sprzężonych par (symetria względem osi OX).
- **Skrajne odchylenia:** Największa dysproporcja dotyczy największych pierwiastków. Pierwiastek dokładny $x = 20$, numerycznie osiąga wartość rzeczywistą rzędu $Re \approx 26$.

Podsumowanie: Fizyczne i analityczne przyczyny błędów

- **Ograniczenia formatu IEEE 754:** Zapis najmniejszych współczynników (np. $a_{19} = -210$) i największych ($a_0 \approx 2.4 \times 10^{18}$) w jednej, ograniczonej rozmiarem pamięci, wymusza obcięcie precyzji (mantysy).

- **Złe uwarunkowanie zadania:** Zgodnie z analizą Wilkinsona, problem wyznaczania pierwiastków tego wielomianu jest ekstremalnie wrażliwy. Zaburzenie pojedynczego współczynnika zaledwie o błąd rzędu 10^{-15} całkowicie deformuje wyniki.

- **Katastrofalne zniesienie (ang. catastrophic cancellation):** W schemacie sumacyjnym naprzemiennie dodawane i odejmowane są liczby rzędu 10^{18}. Operacja odejmowania dwóch bardzo bliskich, lecz obarczonych błędem liczb wielkiej skali, prowadzi do drastycznej kulminacji tego błędu (np. do 27 miliardów).